PERCEIVE–REASON–CODE

Active Perception for Document VQA

Barış Deniz Sağlam

Middle East Technical University • Informatics Institute • PhD Student

ICDAR 2026 DocVQA Challenge • JOINT WINNER • 8–35B PARAMETER TIER

THE METHOD IN ONE TRACE

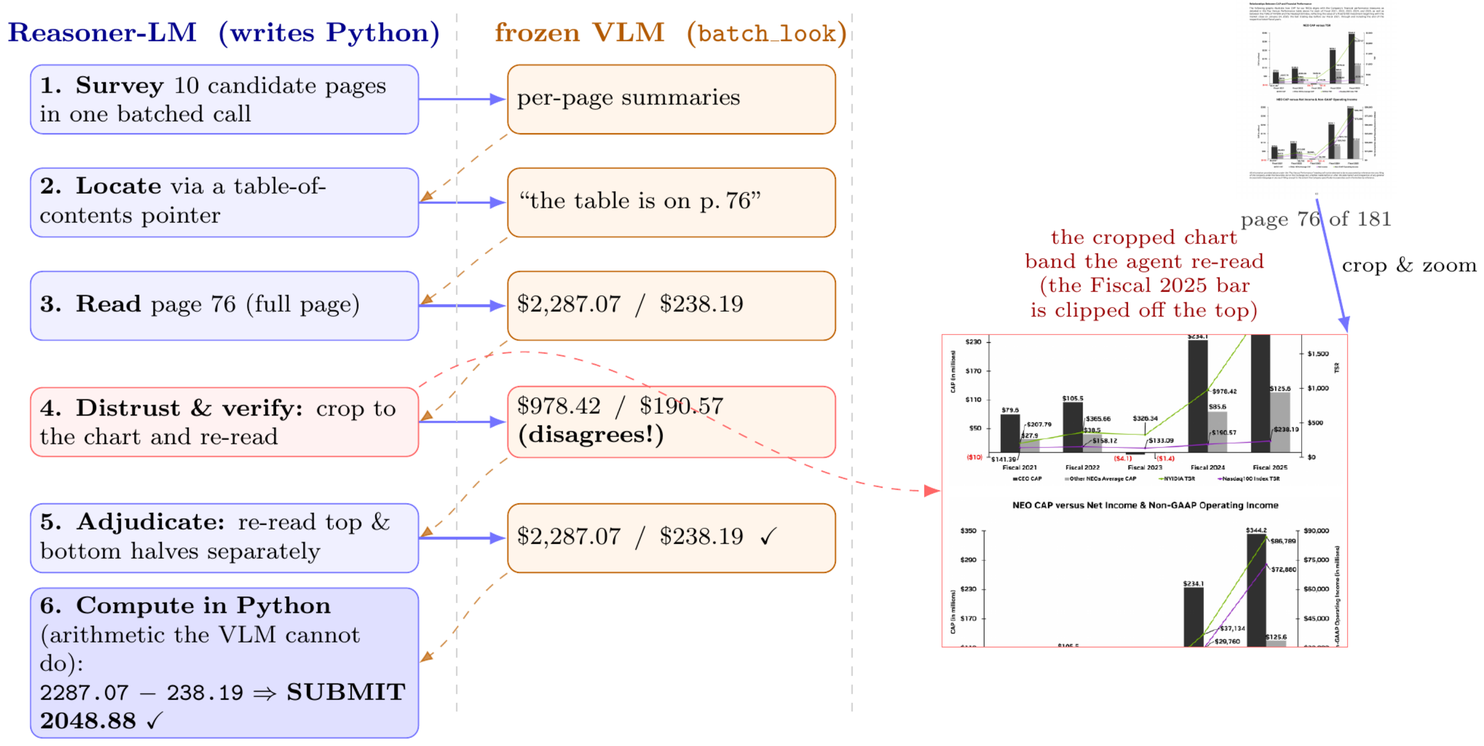

Fig. 1. **Perceive–Reason–Code** on a 181-page report. The reasoner-LM writes Python that calls a *frozen* VLM (`batch_look`) on regions it chooses: it **surveys** pages, **locates** the table, **distrusts** a full-page read, re-reads a tighter **crop** that disagrees, adjudicates, and **computes** the answer in Python — finding *across* pages and reading *within* one. The crop-verify catches a wrong number a single read would have submitted.

ABSTRACT

DocVQA answers questions over documents from a one-page infographic to a 280-page report. General VLMs read every page at fixed resolution and miss fine detail on dense pages. We give a code-capable model (Qwen 3.5 27B) a **Python REPL** and **one perception primitive** — an on-demand VLM call it invokes region by region — so it can crop, zoom, loop, and compute over pages instead of swallowing them whole. With no fine-tuning, it was a **joint winner** of the ICDAR 2026 DocVQA challenge (8–35B tier). Ablations show the lift comes **jointly** from the REPL and the perception call; generality, trajectory format, and added OCR contribute nothing. The binding constraint is a **perception budget**: the models reason fine once they can see the evidence, they just can't resolve a dense page in one look.

THE PROBLEM: TWO KINDS OF SEARCH

Documents are hard along several axes at once:

- **Across pages** — up to **280 pages**; the evidence must first be *found* among them.
- **Within a page** — a page can be **large** (high-resolution) and **dense** (crowded with labels, cells, lines), so the answer region is tiny relative to the page.

Hand a VLM all pages at once and it reads each at fixed resolution with a fixed slice of attention — fine on a sparse page, but on a large, dense one it cannot resolve the answer. The task is both **finding the right page** and **reading the right region on it**.

THE RECIPE

Give a code-capable model a persistent REPL and one perception primitive — an on-demand VLM call on any region of any page — and let it **direct** perception (Fig. 1):

- **Perceive** — a VLM call on a chosen crop.
- **Reason** — in language, in the transcript.
- **Code** — act in Python; observations flow back in.

Builds on Recursive Language Models (Zhang et al., 2025), CodeAct (Wang et al., 2024), and visual programming (Gupta & Kembhavi, 2023; Surís et al., 2023).

RESULTS: WHAT CARRIES THE LIFT

Qwen 3.5 27B as reasoner *and* VLM, DocVQA-2026 val (25 docs / 80 Q), 8 trials, ANLS. Remove one part at a time:

	percep. call	no call
REPL	**41.9%** full	22.3% display()
no REPL	27.2% ReAct	20.9% no scaffold

Both halves are essential — drop either and the score falls to the low-20s. **Three things do not matter:** a general sub-agent (36.7%), append-only vs. compacted trajectory (39.5% vs. 41.9%), and OCR + search on top (36.6%) — all within trial noise.

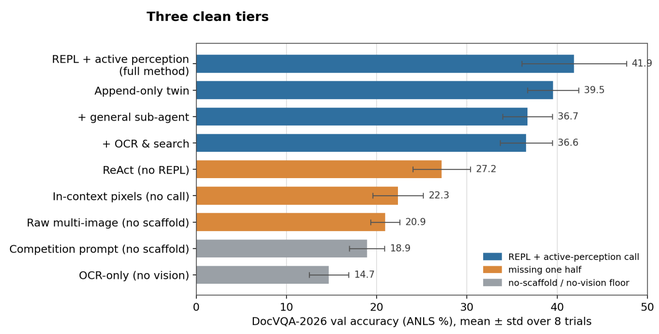

Fig. 2. Three clean tiers: REPL + perception (∼36–42%), missing one half (21–27%), OCR-only floor (15%).

ON THE HELD-OUT TEST SET

System (held-out test)	Score
Qwen 3.5 27B (ours) — tuned	**43.8%**
Qwen 3.5 27B (ours) — general	**39.4%**
Gemini 3 Pro	37.5%
GPT-5.2	35.0%
Gemini 3 Flash	33.8%
GPT-5 Mini	22.5%

Baselines are **bare frontier models** (no scaffold). The **tuned entry** that won the tier added DocVQA-specific prompts + OCR/search (43.8%); the **general** method here strips those and still clears the frontier. Specializing buys a few points; generalizing gives them back.

WHY IT WORKS: A PERCEPTION BUDGET

Swap the eyes for a text channel — OCR of every page plus search, **no vision**. Same scaffold, same reasoner. It falls to **14.7%**, the study floor, and scores **0/10 in all 8 trials** on layout-bound categories (drawings, maps). OCR cannot stand in for looking; the scaffold buys **perception**.

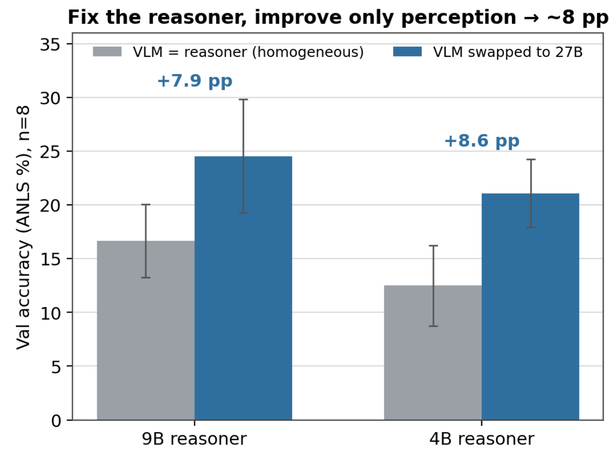

Fig. 3. Hold the reasoner fixed, feed its perception calls a stronger (27B) VLM: accuracy climbs ∼8 pts at both reasoner sizes. Same brain, better eyes — a perception bottleneck.

The lift is a **capacity gate**: the model must be a good-enough coder to drive the REPL. A 31B Gemma clears its ReAct baseline by 14 pts; a 4B Gemma collapses — too weak to write the code.

ADVANTAGE TRACKS VISUAL DENSITY

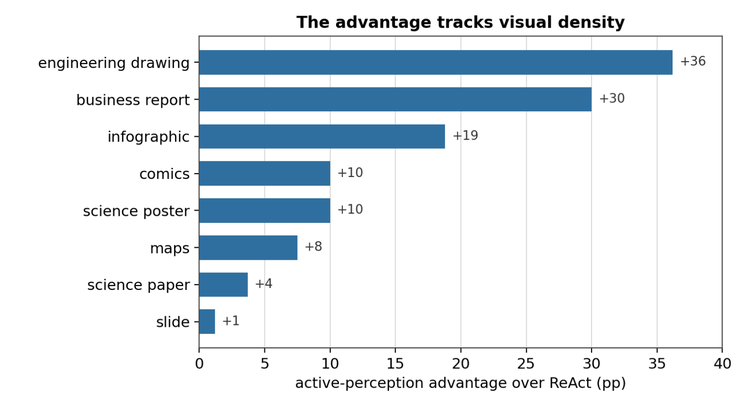

Fig. 4. Advantage over ReAct by category. Largest on dense, structured pages — engineering drawings (+36), business reports (+30), infographics (+19) — smallest on text-linear pages: science papers (+4), slides (+1).

An unspent signal. Over 8 trials the right answer is reached **∼64%** of the time (oracle, pass@8) but reliably produced only **40%** — a ∼24-point gap a learning procedure could capture.

COST & OUTLOOK

Generality costs calls — perception is a region at a time, each a sequential VLM call (∼13 steps/Q); unconstrained recursion can be ruinously expensive (Borchmann et al., 2026). Cheap OCR retrieval or a smaller VLM behind the call are open efficiency levers.

The REPL is a **symbolic substrate** for a neural model — a medium to hold, compose, and compute what its context cannot. That code helps at *test time* is established. The open question: could the symbolic substrate become **part of the model itself** — woven into how it learns and computes, not bolted on at inference? The 24-point oracle gap says there is signal to learn from.

Full technical write-up → scan the code.

`barisdeniz.is-a.dev`

`github.com/bdsaglam/docvqa`

REFERENCES

Zhang, A.L., Kraska, T., Khattab, O. (2025). *Recursive Language Models.* arXiv:2512.24601.

Wang, X. et al. (2024). *Executable Code Actions Elicit Better LLM Agents (CodeAct).* ICML.

Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in LMs.* ICLR.

Gupta, T., Kembhavi, A. (2023). *Visual Programming (VisProg).* CVPR.

Surís, D., Menon, S., Vondrick, C. (2023). *ViperGPT.* ICCV.

Zheng, C. et al. (2025). *DeepEyes: Thinking with Images via RL.* arXiv:2505.14362.

Borchmann, Ł. et al. (2026). *Strategic Navigation or Stochastic Search? (MADQA).* arXiv:2603.12180.

Mathew, M., Karatzas, D., Jawahar, C.V. (2021). *DocVQA.* WACV.